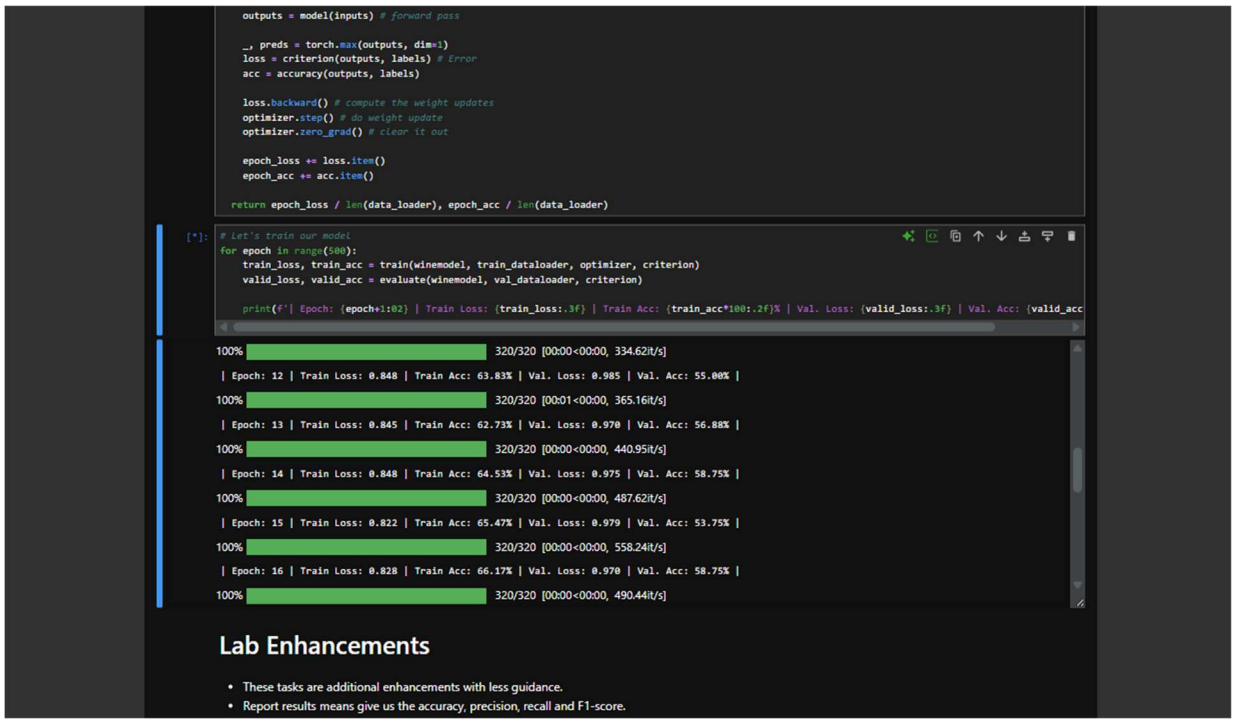
Justin Gould

Deep Learning on Cloud Platforms

Lab 1

October 2, 2026

Code running:



```
    outputs = model(inputs) # forward pass

    _, preds = torch.max(outputs, dim=1)
    loss = criterion(outputs, labels) # Error
    acc = accuracy(outputs, labels)

    loss.backward() # compute the weight updates
    optimizer.step() # do weight update
    optimizer.zero_grad() # clear it out

    epoch_loss += loss.item()
    epoch_acc += acc.item()

  return epoch_loss / len(data_loader), epoch_acc / len(data_loader)
```

```
[*]: # Let's train our model
for epoch in range(500):
    train_loss, train_acc = train(winemodel, train_dataloader, optimizer, criterion)
    valid_loss, valid_acc = evaluate(winemodel, val_dataloader, criterion)

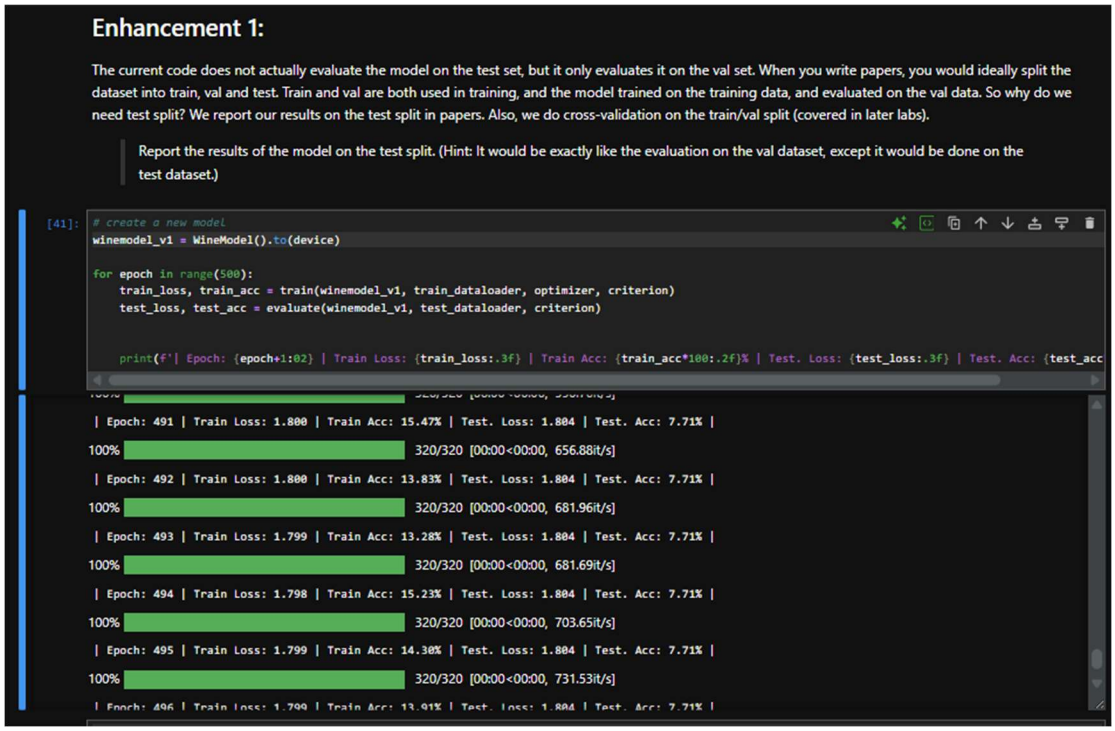
    print(f'| Epoch: {epoch+1:02} | Train Loss: {train_loss:.3f} | Train Acc: {train_acc*100:.2f}% | Val. Loss: {valid_loss:.3f} | Val. Acc: {valid_acc
```

```
100% 320/320 [00:00<00:00, 334.62it/s]
| Epoch: 12 | Train Loss: 0.848 | Train Acc: 63.83% | Val. Loss: 0.985 | Val. Acc: 55.00% |
100% 320/320 [00:01<00:00, 365.16it/s]
| Epoch: 13 | Train Loss: 0.845 | Train Acc: 62.73% | Val. Loss: 0.970 | Val. Acc: 56.88% |
100% 320/320 [00:00<00:00, 440.95it/s]
| Epoch: 14 | Train Loss: 0.848 | Train Acc: 64.53% | Val. Loss: 0.975 | Val. Acc: 58.75% |
100% 320/320 [00:00<00:00, 487.62it/s]
| Epoch: 15 | Train Loss: 0.822 | Train Acc: 65.47% | Val. Loss: 0.979 | Val. Acc: 53.75% |
100% 320/320 [00:00<00:00, 558.24it/s]
| Epoch: 16 | Train Loss: 0.828 | Train Acc: 66.17% | Val. Loss: 0.970 | Val. Acc: 58.75% |
100% 320/320 [00:00<00:00, 490.44it/s]
```

Lab Enhancements

- These tasks are additional enhancements with less guidance.
- Report results means give us the accuracy, precision, recall and F1-score.

Enhancement 1:



Enhancement 1:

The current code does not actually evaluate the model on the test set, but it only evaluates it on the val set. When you write papers, you would ideally split the dataset into train, val and test. Train and val are both used in training, and the model trained on the training data, and evaluated on the val data. So why do we need test split? We report our results on the test split in papers. Also, we do cross-validation on the train/val split (covered in later labs).

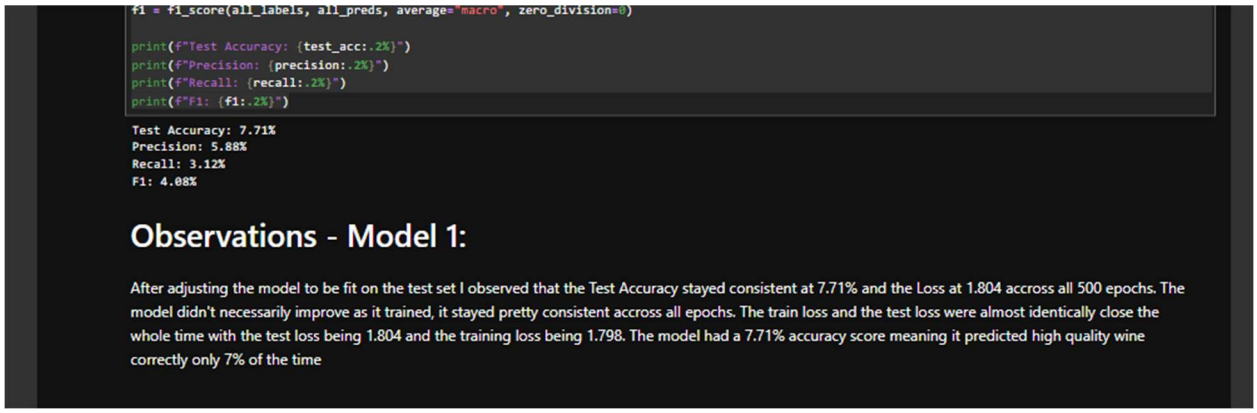
> Report the results of the model on the test split. (Hint: It would be exactly like the evaluation on the val dataset, except it would be done on the test dataset.)

[41]:
```
# create a new model
winemodel_v1 = WineModel().to(device)

for epoch in range(500):
    train_loss, train_acc = train(winemodel_v1, train_dataloader, optimizer, criterion)
    test_loss, test_acc = evaluate(winemodel_v1, test_dataloader, criterion)


    print(f'| Epoch: {epoch+1:02} | Train Loss: {train_loss:.3f} | Train Acc: {train_acc*100:.2f}% | Test. Loss: {test_loss:.3f} | Test. Acc: {test_acc
```

```
| Epoch: 491 | Train Loss: 1.800 | Train Acc: 15.47% | Test. Loss: 1.804 | Test. Acc: 7.71% |
100% 320/320 [00:00<00:00, 656.88it/s]
| Epoch: 492 | Train Loss: 1.800 | Train Acc: 13.83% | Test. Loss: 1.804 | Test. Acc: 7.71% |
100% 320/320 [00:00<00:00, 681.96it/s]
| Epoch: 493 | Train Loss: 1.799 | Train Acc: 13.28% | Test. Loss: 1.804 | Test. Acc: 7.71% |
100% 320/320 [00:00<00:00, 681.69it/s]
| Epoch: 494 | Train Loss: 1.798 | Train Acc: 15.23% | Test. Loss: 1.804 | Test. Acc: 7.71% |
100% 320/320 [00:00<00:00, 703.65it/s]
| Epoch: 495 | Train Loss: 1.799 | Train Acc: 14.30% | Test. Loss: 1.804 | Test. Acc: 7.71% |
100% 320/320 [00:00<00:00, 731.53it/s]
| Epoch: 496 | Train Loss: 1.799 | Train Acc: 13.91% | Test. Loss: 1.804 | Test. Acc: 7.71% |
```



```
f1 = f1_score(all_labels, all_preds, average="macro", zero_division=0)

print(f"Test Accuracy: {test_acc:.2%}")
print(f"Precision: {precision:.2%}")
print(f"Recall: {recall:.2%}")
print(f"F1: {f1:.2%}")
```

```
Test Accuracy: 7.71%
Precision: 5.88%
Recall: 3.12%
F1: 4.08%
```

Observations - Model 1:

After adjusting the model to be fit on the test set I observed that the Test Accuracy stayed consistent at 7.71% and the Loss at 1.804 accross all 500 epochs. The model didn't necessarily improve as it trained, it stayed pretty consistent accross all epochs. The train loss and the test loss were almost identically close the whole time with the test loss being 1.804 and the training loss being 1.798. The model had a 7.71% accuracy score meaning it predicted high quality wine correctly only 7% of the time

- **Enhancement 2:**

Enhancement 2:

> Increase the number of epochs (and maybe the learning rate). Does the accuracy on the test set increase? Is there a significant difference between the test accuracy and the train accuracy? If yes, why?

[44]:
```
# increase the Learning rate
optimizer1 = torch.optim.AdamW(winemodel.parameters(), lr=1e-2, weight_decay=1e-2)

# create a new model
winemodel_v2 = WineModel().to(device)
```

[45]:
```
# increased the epoch's to 550
for epoch in range(550):
    train_loss, train_acc = train(winemodel_v2, train_dataloader, optimizer1, criterion)
    test_loss, test_acc = evaluate(winemodel_v2, test_dataloader, criterion)


    print(f'| Epoch: {epoch+1:02} | Train Loss: {train_loss:.3f} | Train Acc: {train_acc*100:.2f}% | Test. Loss: {test_loss:.3f} | Test. Acc: {test_acc
```

```
| Epoch: 545 | Train Loss: 1.759 | Train Acc: 16.80% | Test. Loss: 1.754 | Test. Acc: 21.88% |
100% 320/320 [00:00<00:00, 455.49it/s]
| Epoch: 546 | Train Loss: 1.759 | Train Acc: 17.27% | Test. Loss: 1.754 | Test. Acc: 21.88% |
100% 320/320 [00:00<00:00, 427.51it/s]
| Epoch: 547 | Train Loss: 1.757 | Train Acc: 17.97% | Test. Loss: 1.754 | Test. Acc: 21.88% |
100% 320/320 [00:00<00:00, 434.84it/s]
| Epoch: 548 | Train Loss: 1.759 | Train Acc: 17.58% | Test. Loss: 1.754 | Test. Acc: 21.88% |
100% 320/320 [00:00<00:00, 454.59it/s]
| Epoch: 549 | Train Loss: 1.758 | Train Acc: 17.11% | Test. Loss: 1.754 | Test. Acc: 21.88% |
100% 320/320 [00:00<00:00, 445.74it/s]
| Epoch: 550 | Train Loss: 1.758 | Train Acc: 18.12% | Test. Loss: 1.754 | Test. Acc: 21.88% |
```

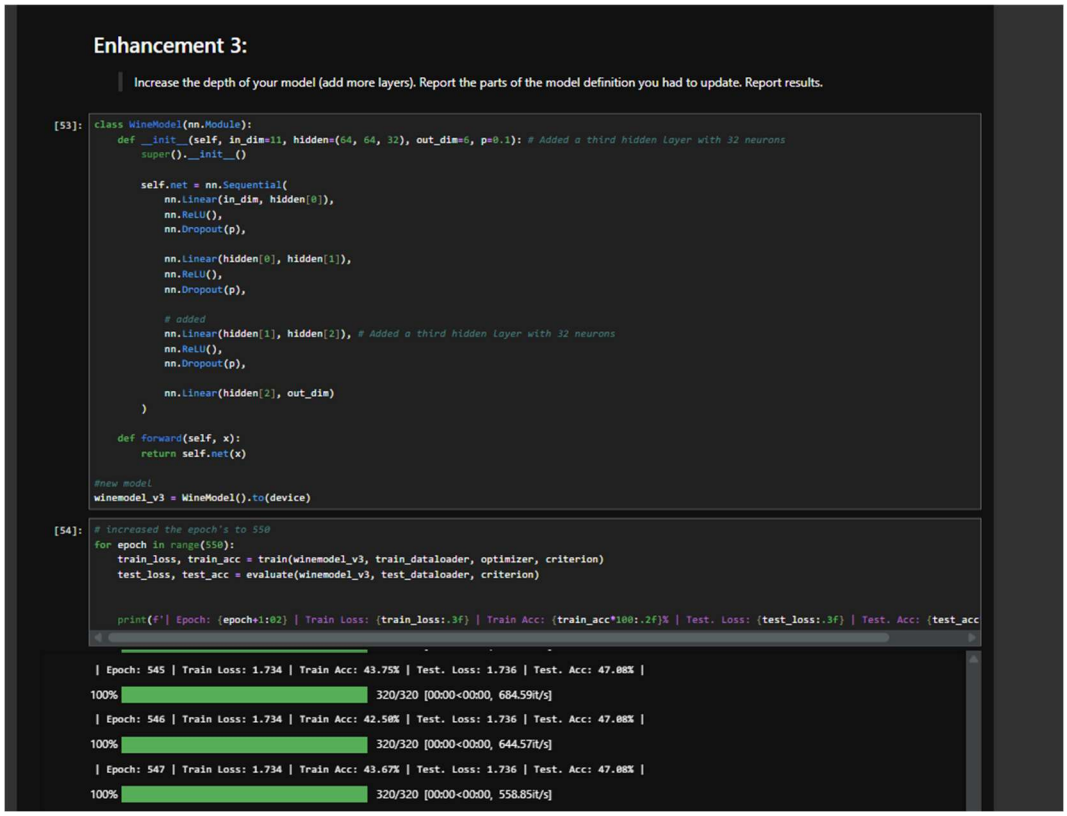
```
print(f"Test Accuracy: {test_acc:.2%}")
print(f"Precision: {precision:.2%}")
print(f"Recall: {recall:.2%}")
print(f"F1: {f1:.2%}")
```

```
Test Accuracy: 21.88%
Precision: 15.16%
Recall: 23.78%
F1: 11.85%
```

Observations - Model 2:

Much like model 1, model 2 did not change very much in terms of the accuracy and loss throughout each epoch. The test accuracy in this model was much higher at 21.88% showing that this model correctly predicted wine quality better than model 1. **Increasing the learning rate of the model proved be an effective way to enhance the model**

Enhancement 3:



Enhancement 3:

> Increase the depth of your model (add more layers). Report the parts of the model definition you had to update. Report results.

[53]:
```
class WineModel(nn.Module):
    def __init__(self, in_dim=11, hidden=(64, 64, 32), out_dim=6, p=0.1): # Added a third hidden layer with 32 neurons
        super().__init__()

        self.net = nn.Sequential(
            nn.Linear(in_dim, hidden[0]),
            nn.ReLU(),
            nn.Dropout(p),

            nn.Linear(hidden[0], hidden[1]),
            nn.ReLU(),
            nn.Dropout(p),

            # added
            nn.Linear(hidden[1], hidden[2]), # Added a third hidden layer with 32 neurons
            nn.ReLU(),
            nn.Dropout(p),

            nn.Linear(hidden[2], out_dim)
        )

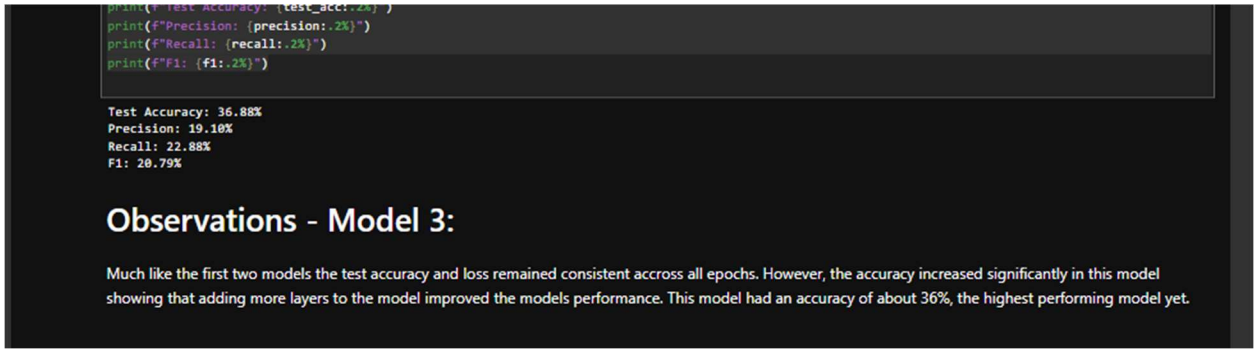
    def forward(self, x):
        return self.net(x)

#new model
winemodel_v3 = WineModel().to(device)
```

[54]:
```
# increased the epoch's to 550
for epoch in range(550):
    train_loss, train_acc = train(winemodel_v3, train_dataloader, optimizer, criterion)
    test_loss, test_acc = evaluate(winemodel_v3, test_dataloader, criterion)


    print(f'| Epoch: {epoch+1:02} | Train Loss: {train_loss:.3f} | Train Acc: {train_acc*100:.2f}% | Test. Loss: {test_loss:.3f} | Test. Acc: {test_acc
```

```
| Epoch: 545 | Train Loss: 1.734 | Train Acc: 43.75% | Test. Loss: 1.736 | Test. Acc: 47.08% |
100% 320/320 [00:00<00:00, 684.59it/s]
| Epoch: 546 | Train Loss: 1.734 | Train Acc: 42.50% | Test. Loss: 1.736 | Test. Acc: 47.08% |
100% 320/320 [00:00<00:00, 644.57it/s]
| Epoch: 547 | Train Loss: 1.734 | Train Acc: 43.67% | Test. Loss: 1.736 | Test. Acc: 47.08% |
100% 320/320 [00:00<00:00, 558.85it/s]
```



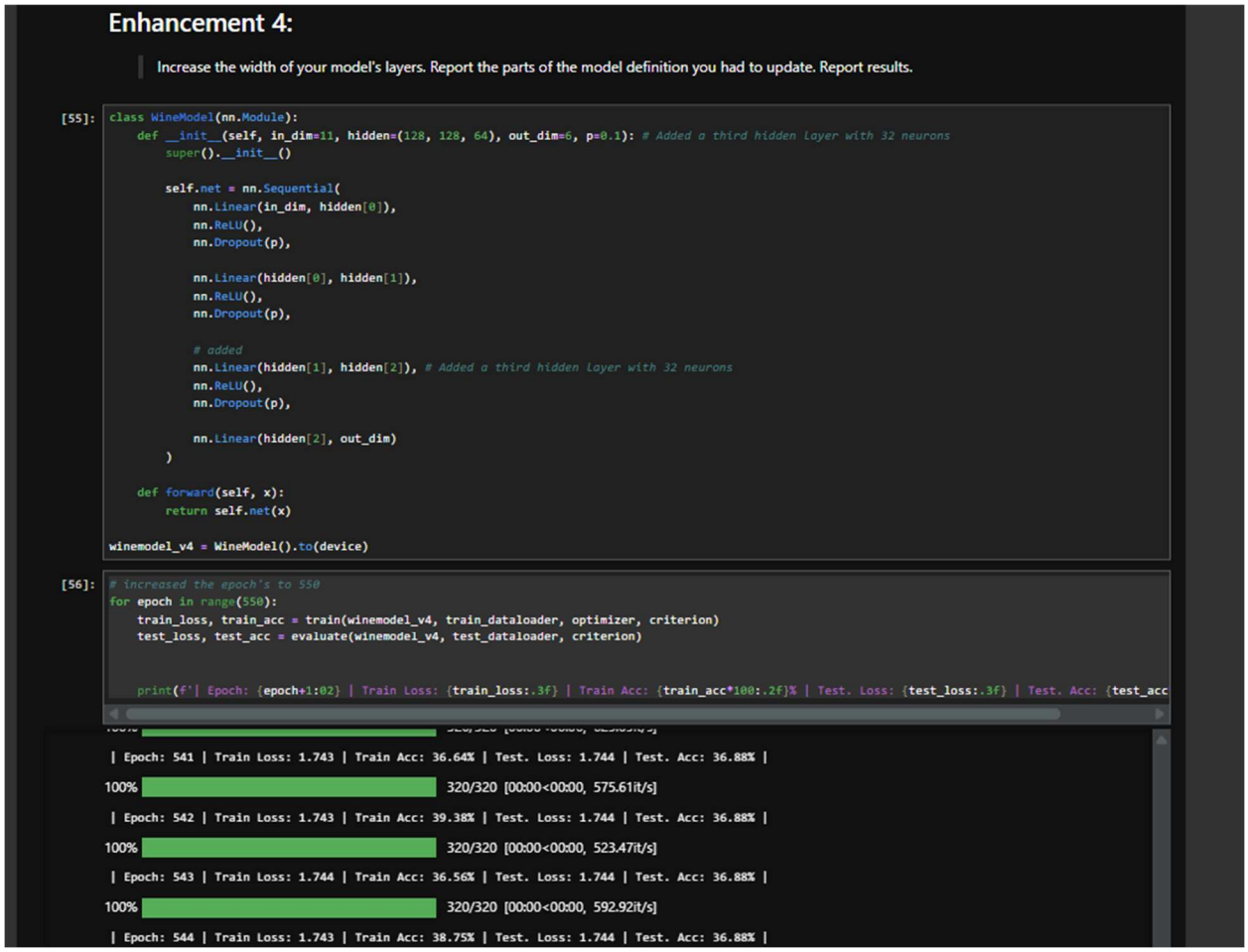
```
print(f"Test Accuracy: {test_acc:.2%}")
print(f"Precision: {precision:.2%}")
print(f"Recall: {recall:.2%}")
print(f"F1: {f1:.2%}")
```

```
Test Accuracy: 36.88%
Precision: 19.10%
Recall: 22.88%
F1: 20.79%
```

Observations - Model 3:

Much like the first two models the test accuracy and loss remained consistent accross all epochs. However, the accuracy increased significantly in this model showing that adding more layers to the model improved the models performance. This model had an accuracy of about 36%, the highest performing model yet.

Enhancement 4:



Enhancement 4:

> Increase the width of your model's layers. Report the parts of the model definition you had to update. Report results.

[55]:
```
class WineModel(nn.Module):
    def __init__(self, in_dim=11, hidden=(128, 128, 64), out_dim=6, p=0.1): # Added a third hidden layer with 32 neurons
        super().__init__()

        self.net = nn.Sequential(
            nn.Linear(in_dim, hidden[0]),
            nn.ReLU(),
            nn.Dropout(p),

            nn.Linear(hidden[0], hidden[1]),
            nn.ReLU(),
            nn.Dropout(p),

            # added
            nn.Linear(hidden[1], hidden[2]), # Added a third hidden layer with 32 neurons
            nn.ReLU(),
            nn.Dropout(p),

            nn.Linear(hidden[2], out_dim)
        )

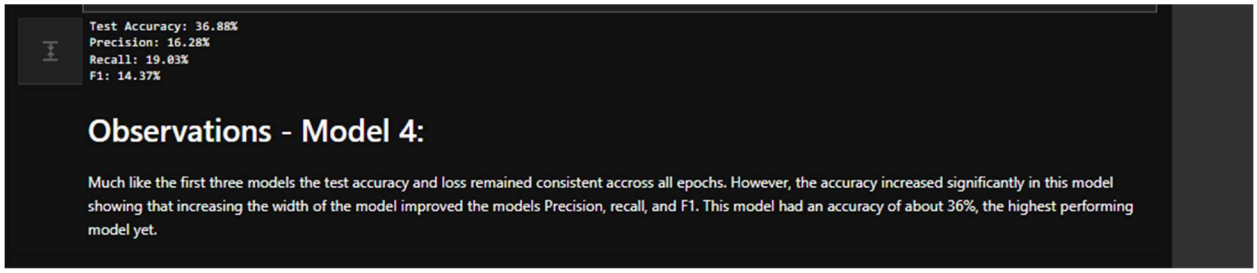
    def forward(self, x):
        return self.net(x)

winemodel_v4 = WineModel().to(device)
```

[56]:
```
# increased the epoch's to 550
for epoch in range(550):
    train_loss, train_acc = train(winemodel_v4, train_dataloader, optimizer, criterion)
    test_loss, test_acc = evaluate(winemodel_v4, test_dataloader, criterion)


    print(f'| Epoch: {epoch+1:02} | Train Loss: {train_loss:.3f} | Train Acc: {train_acc*100:.2f}% | Test. Loss: {test_loss:.3f} | Test. Acc: {test_acc
```

```
| Epoch: 541 | Train Loss: 1.743 | Train Acc: 36.64% | Test. Loss: 1.744 | Test. Acc: 36.88% |
100% 320/320 [00:00<00:00, 575.61it/s]
| Epoch: 542 | Train Loss: 1.743 | Train Acc: 39.38% | Test. Loss: 1.744 | Test. Acc: 36.88% |
100% 320/320 [00:00<00:00, 523.47it/s]
| Epoch: 543 | Train Loss: 1.744 | Train Acc: 36.56% | Test. Loss: 1.744 | Test. Acc: 36.88% |
100% 320/320 [00:00<00:00, 592.92it/s]
| Epoch: 544 | Train Loss: 1.743 | Train Acc: 38.75% | Test. Loss: 1.744 | Test. Acc: 36.88% |
```



```
Test Accuracy: 36.88%
Precision: 16.28%
Recall: 19.03%
F1: 14.37%
```

Observations - Model 4:

Much like the first three models the test accuracy and loss remained consistent accross all epochs. However, the accuracy increased significantly in this model showing that increasing the width of the model improved the models Precision, recall, and F1. This model had an accuracy of about 36%, the highest performing model yet.

Enhancement 5:

Enhancement 5:

> Choose a new dataset from the list below. Search the Internet and download your chosen dataset (many of them could be available on kaggle). Adapt your model to your dataset. Train your model and record your results.

- cancer_dataset - Breast cancer dataset.
- crab_dataset - Crab gender dataset.
- glass_dataset - Glass chemical dataset.
- iris_dataset - Iris flower dataset.
- ovarian_dataset - Ovarian cancer dataset.
- thyroid_dataset - Thyroid function dataset.

[95]:
```
import pandas as pd
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import precision_score, recall_score, f1_score
from tqdm.notebook import tqdm

# Load dataset
data_df1 = pd.read_csv('Iris.csv')

# Encode species into numerical labels (0, 1, 2)
le = LabelEncoder()
data_df1['Species'] = le.fit_transform(data_df1['Species'])
```

[97]:
```
class IrisDataset(Dataset):
    def __init__(self, data_df1):
        self.features = []
        self.labels = []

        for _, i in data_df1.iterrows():
            self.features.append([
                i['SepalLengthCm'],
                i['SepalWidthCm'],
                i['PetalLengthCm'],
                i['PetalWidthCm']
            ])

            self.labels.append(i['Species'])

    def __len__(self):
        return len(self.features)

    def __getitem__(self, idx):
        features = torch.tensor(self.features[idx], dtype=torch.float32)
        labels = torch.tensor(self.labels[idx], dtype=torch.long)

        return {'labels': labels, 'features': features}
```

[105]:
```
for epoch in range(550):
    train_loss, train_acc = train(
        irismodel, train_dataloader, optimizer, criterion
    )

    test_loss, test_acc = evaluate(
        irismodel, test_dataloader, criterion
    )

    print(
        f'| Epoch: {epoch+1:02} '
        f'| Train Loss: {train_loss:.3f} '
        f'| Train Acc: {train_acc*100:.2f}% '
        f'| Test Loss: {test_loss:.3f} '
        f'| Test Acc: {test_acc*100:.2f}% |'
    )
```

```
| Epoch: 544 | Train Loss: 0.006 | Train Acc: 100.00% | Test Loss: 1.070 | Test Acc: 81.25% |
100% 30/30 [00:00<00:00, 602.14it/s]
| Epoch: 545 | Train Loss: 0.006 | Train Acc: 100.00% | Test Loss: 0.521 | Test Acc: 93.75% |
100% 30/30 [00:00<00:00, 579.65it/s]
| Epoch: 546 | Train Loss: 0.011 | Train Acc: 99.17% | Test Loss: 1.503 | Test Acc: 81.25% |
100% 30/30 [00:00<00:00, 569.01it/s]
| Epoch: 547 | Train Loss: 0.124 | Train Acc: 95.00% | Test Loss: 0.993 | Test Acc: 81.25% |
100% 30/30 [00:00<00:00, 637.83it/s]
| Epoch: 548 | Train Loss: 0.013 | Train Acc: 99.17% | Test Loss: 0.398 | Test Acc: 93.75% |
100% 30/30 [00:00<00:00, 599.93it/s]
| Epoch: 549 | Train Loss: 0.041 | Train Acc: 98.33% | Test Loss: 0.991 | Test Acc: 81.25% |
```

[107]:
```
all_preds = []
```

```
) / len(all_labels)

print(f'Test Accuracy: {test_accuracy:.2%}')
print(f'Precision: {precision:.2%}')
print(f'Recall: {recall:.2%}')
print(f'F1: {f1:.2%}')
```

```
Test Accuracy: 80.00%
Precision: 66.67%
Recall: 56.67%
F1: 60.78%
```

Observations:

This model predicted the type of Iris based off the length and width of the pedal and the sepal. It is a very effective model predicting correctly 8-% of the time with a precion of category of 66%.